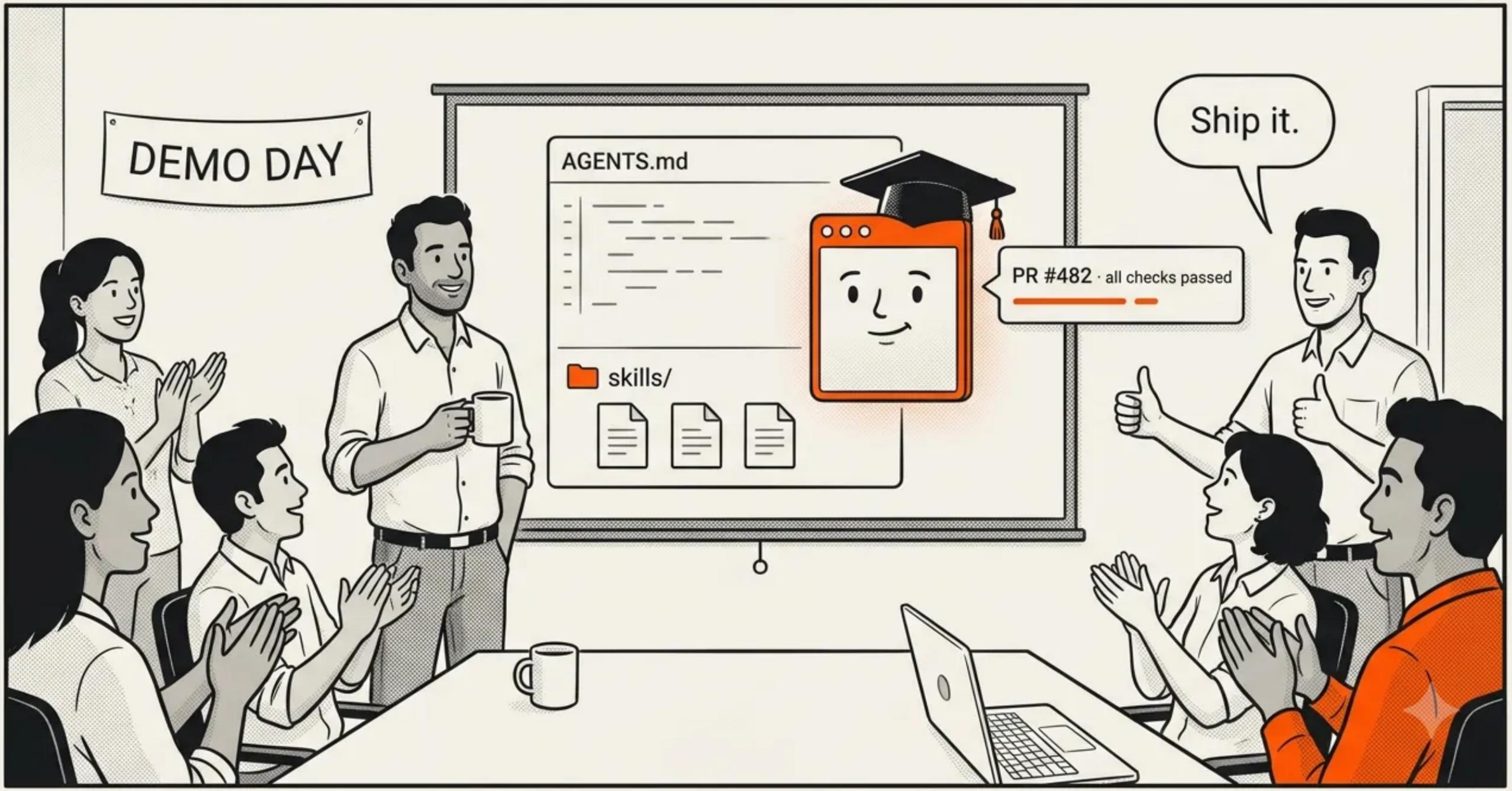
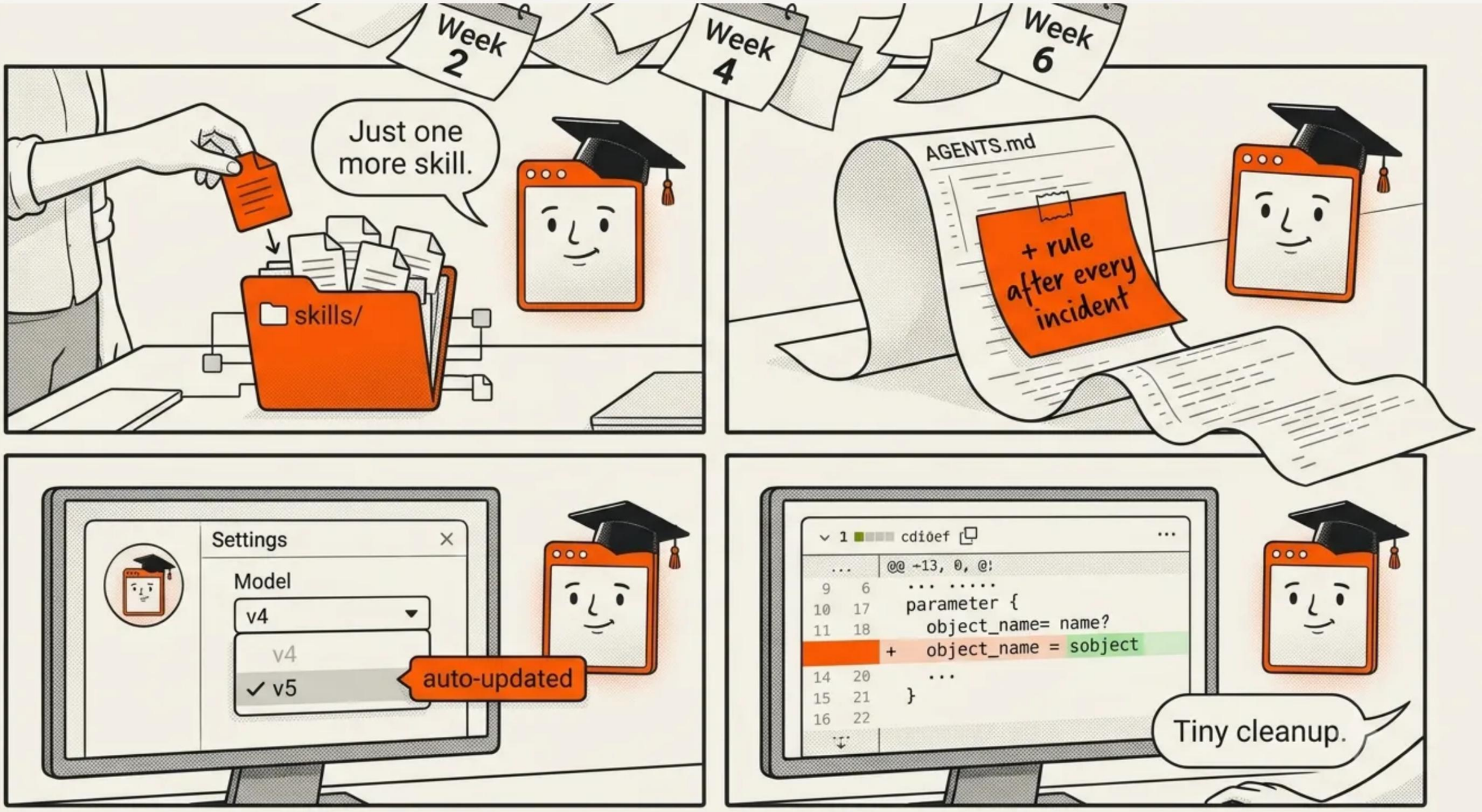
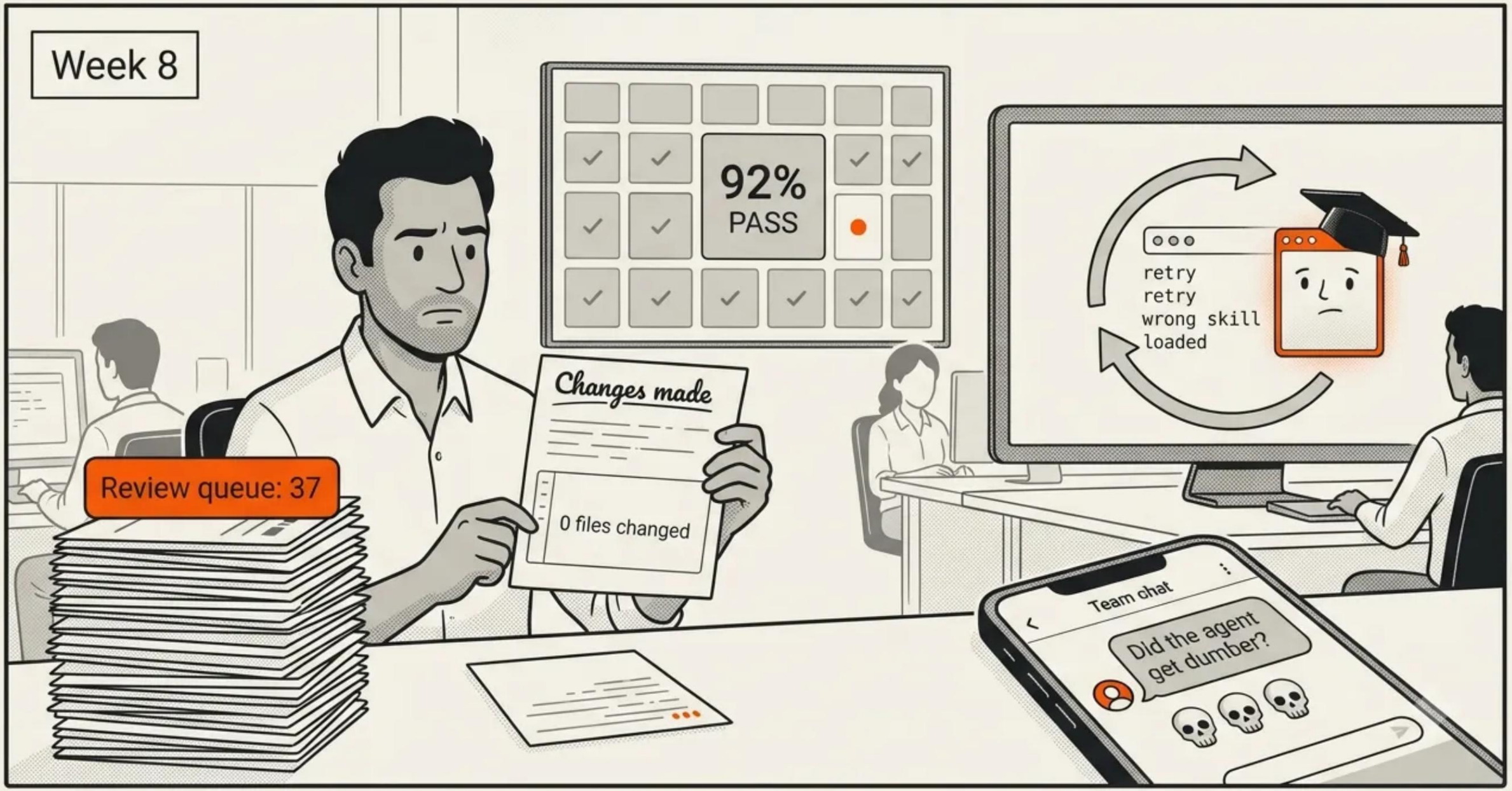
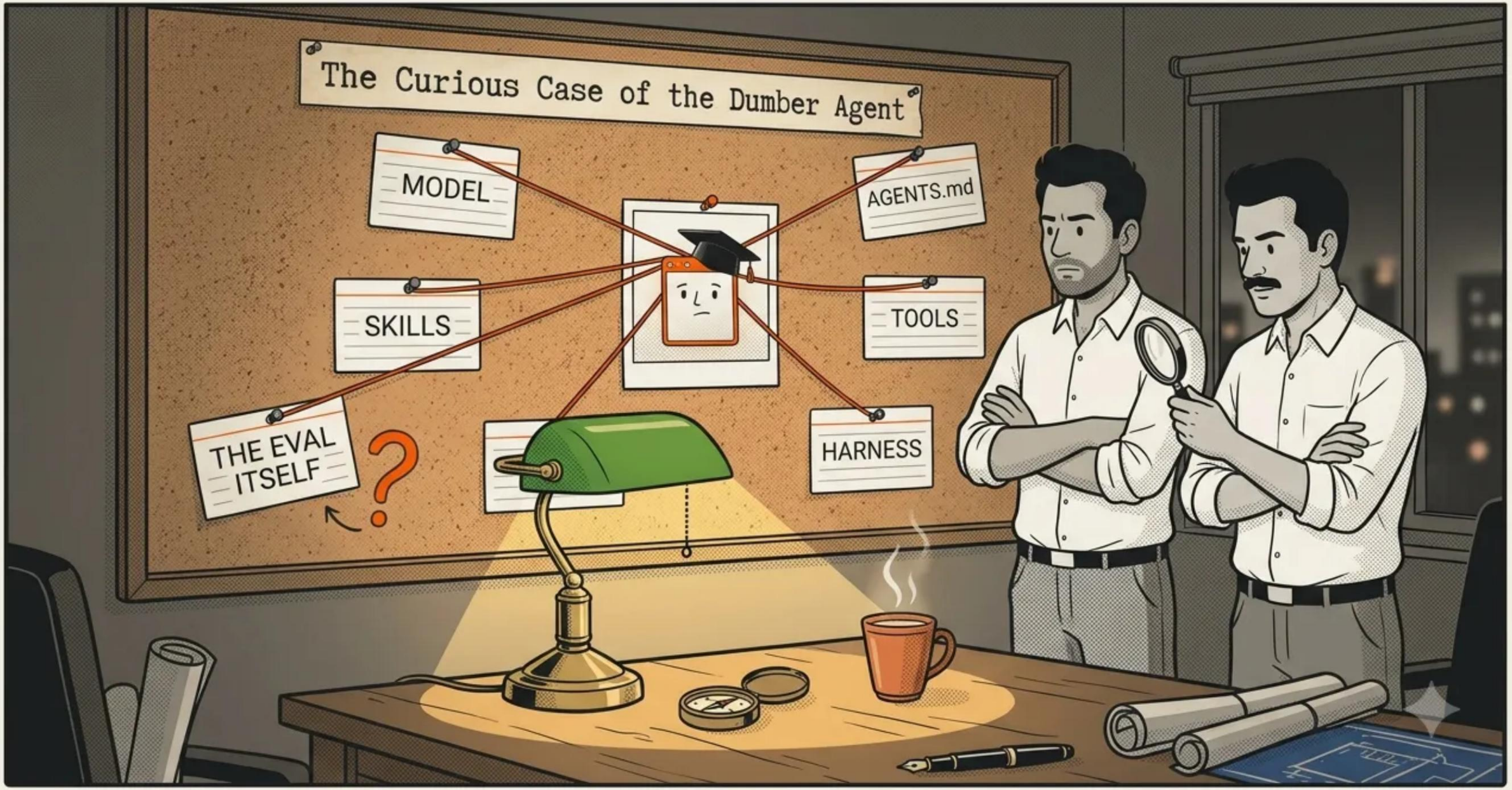

DEMO DAY
AGENTS.md
skills/
PR #482 · all checks passed
Ship it.


Week 2
Week 4
Week 6
Just one more skill.
skills/
AGENTS.md
+ rule after every incident
Settings
Model
v4
v4
✓ v5
auto-updated
1 cdi0ef
@@ -13, 0, @!
parameter {
object_name= name?
object_name = sobject
}
Tiny cleanup.


Week 8
92%
PASS
Changes made
0 files changed
Review queue: 37
retry
retry
wrong skill
loaded
Team chat
Did the agent
get dumber?


The Curious Case of the Dumber Agent
MODEL
AGENTS.md
SKILLS
TOOLS
THE EVAL ITSELF
?
HARNESS

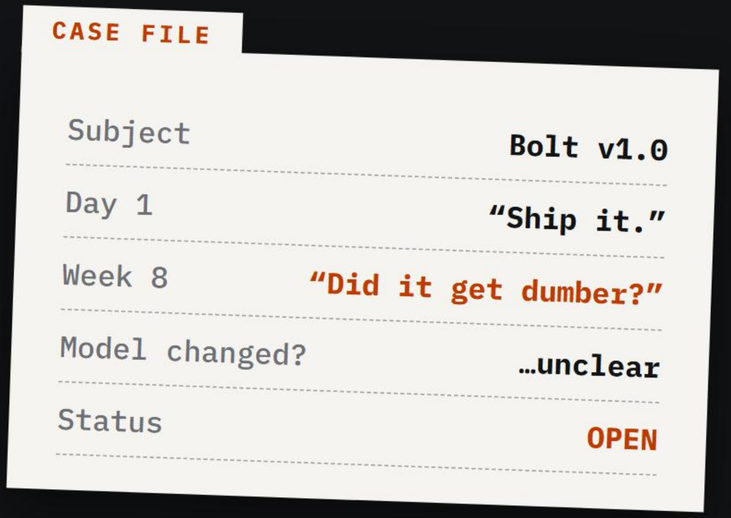
CS6.403: AUTONOMOUS SOFTWARE ENGINEERING (AUTOSE) · LECTURE 10

The Curious Case of the Dumber Agent

Evaluating and evolving the skills, decisions and architectures around AI agents.

GUEST LECTURE

Tezan Sahu





Tezan Sahu

A LITTLE ABOUT ME

MICROSOFT IDC · 2021 – PRESENT

Senior Applied Scientist

M365 Copilot Extensibility

10+ US patents · papers on agent evaluation

IIT BOMBAY · 2017 – 21

Institute Rank 2

Mechanical Engineering, 9.90 CPI

Minor in Computer Science & Engineering

AUTHOR

“Beyond Code”

National bestseller

Low-Pass Filter, an AI newsletter

SPEAKER & MENTOR

TEDx & PyConf

speaker

50+ talks · 25,000+ learners

Views are my own. Internal systems appear only as generalised patterns.

WHERE THIS LECTURE PICKS UP

You already know how to build the parts

SKILLS · L04

SKILL.md, progressive disclosure, “the description is the trigger”

SPECS · L05

AGENTS.md as the constitution, Spec Kit, Kiro, EARS

HARNESSES · L07

Nested loops, and the parts around them, including **verification**

PATTERNS · L08

Reflection, voting, debate, sub-agents, skill libraries, human approval

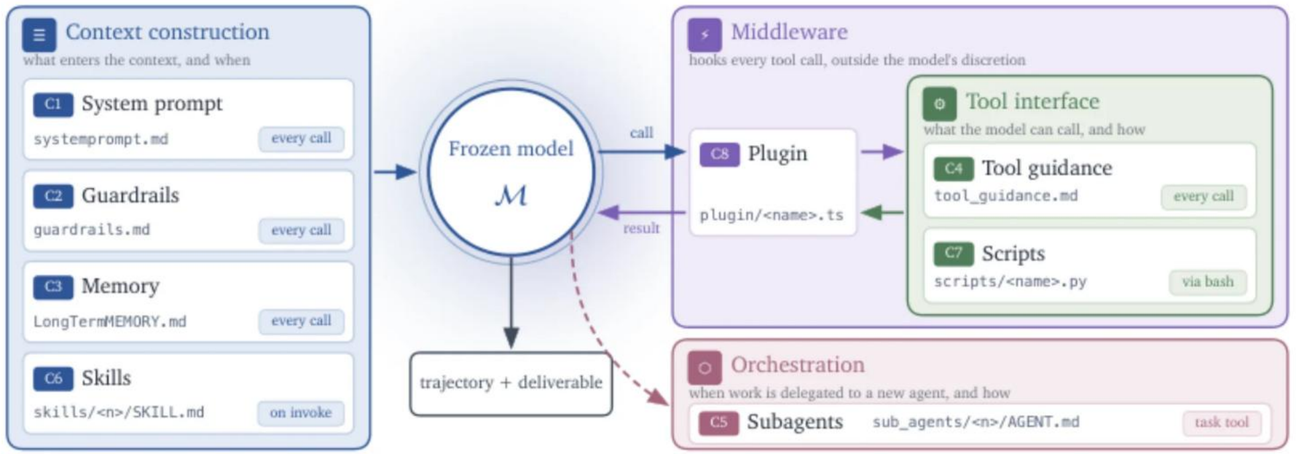
> *Skill library evolution, cons: quality is hard to manage · outdated skills · skills break when a dependency changes · **no way to compare a stored skill with a fresh generation***

Today: those open problems, investigated. Why agents built from these parts get dumber, and how they can get smarter.

Quoted from Lecture 08, “Patterns for Building SE Agents” (Skill Library Evolution)

THE ONE EQUATION FOR TODAY

Agent = model + harness



A frozen model, and everything around it as **editable files**

You rarely change the **model**.
Someone else trains it.

You change the **harness** every day: prompts, skills, memory, tools, sub-agents, hooks.

Every suspect on our case board lives **here**.

Figure: "Harness²: Recursive Agent Harnessing for an Open World", Google (2026) · see also harness-engineering survey (2026)

TODAY'S INVESTIGATION

Six suspects, and a verdict



SUSPECT 01

The context we wrote

Did all those instructions and skills make it smarter, or heavier?

A QUESTION MOST TEAMS SKIP

Does an AGENTS.md or architecture doc actually help a coding agent?

VENDORS RECOMMEND THEM

Model providers suggest repo overviews and context files

ALMOST EVERY REPO HAS ONE

AGENTS.md, CLAUDE.md, copilot-instructions, rules files

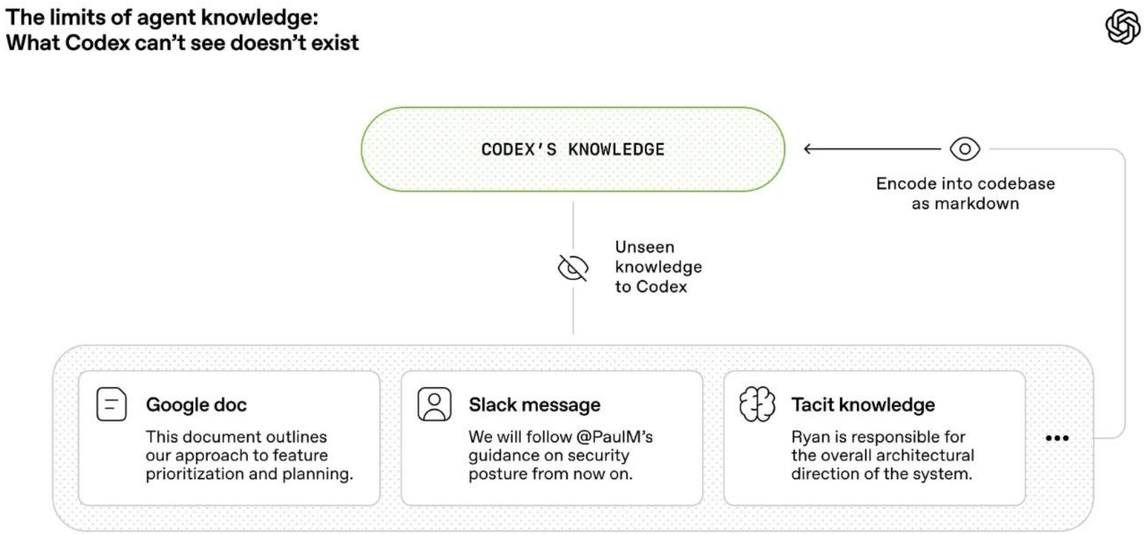
FEW TEAMS MEASURE THEM

Added on intuition, kept forever, rarely tested

Show of hands: who expects the architecture doc to clearly help?

WHAT PRACTITIONERS RECOMMEND

“Give the agent a map, not a 1,000-page manual”



If it isn't in the repo, **it doesn't exist** for the agent

~1M

lines of code in 5 months, **0** written by hand

~1,500

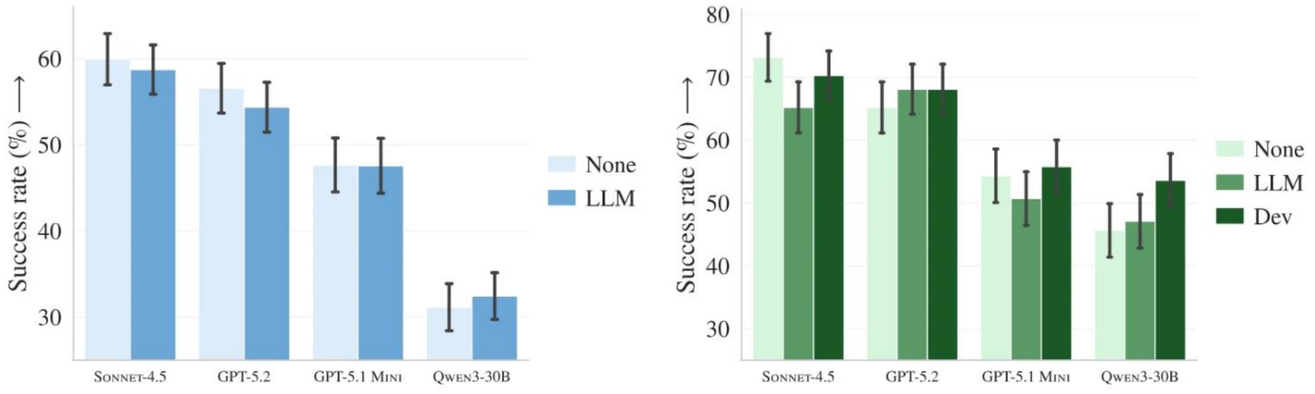
merged PRs, driven by 3 engineers (later 7)

A **~100-line AGENTS.md** as a table of contents, pointing to an ARCHITECTURE.md and a **docs/** system of record

Lopopolo, “Harness engineering: leveraging Codex in an agent-first world”, OpenAI (2026)

WHAT THE FIRST CAREFUL STUDY FOUND

Context files didn't raise success. They raised cost.



Success rate for 4 models: **no file**, an **LLM-written** file, a **developer-written** file · SWE-bench (left), real repos with committed files (right)

+20%

average inference cost when a context file is present

Agents **followed** the instructions. They explored and tested more.

Repository overviews didn't help. Non-standard practices did.

Gloaguen et al., "Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?", ETH Zürich (2026)

SO WHO'S RIGHT?

Both. What helps is the right context, enforced.

TENDS NOT TO HELP

- Overviews the agent could read from the code
- Long, “everything is important” files
- Rules nobody checks, so they quietly go stale

TENDS TO HELP

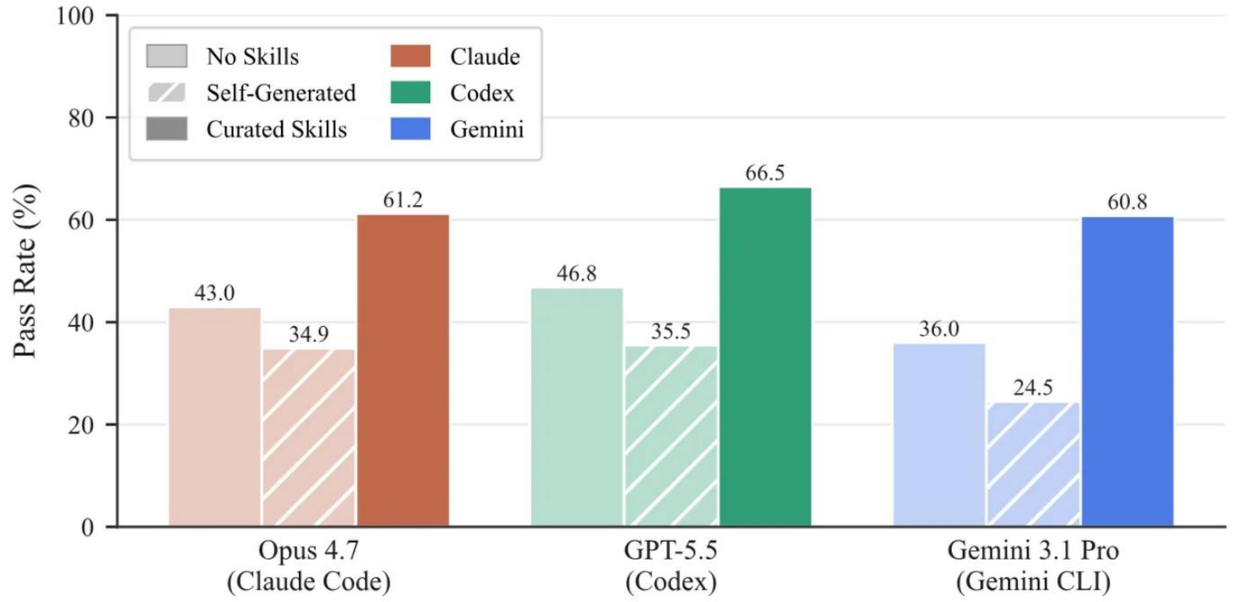
- Things the agent can't guess: non-standard conventions
- A short map that points to deeper docs
- Rules promoted into linters and tests

A context file is a hypothesis about what the agent is missing.
Like any hypothesis, it needs a test.

Synthesis of Gloaguen et al. (2026) and OpenAI, “Harness engineering” (2026)

THE SAME QUESTION, FOR SKILLS

Curated skills help. Skills a model writes for itself hurt.



No skills · **self-generated** (hatched) · **curated**, on three model–harness pairs

33.9 → 50.5%

average pass rate with curated skills, across 18 model–harness pairs (**+4 to +26** by pair)

–8 to –12

points when the model writes its own skill from its priors

≤ **3 focused modules** beat bundles.
What self-written skills lack is **evidence**.

Li et al., "SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks" (v4, 2026), incl. the self-generated-skills ablation

WHAT WE SEE IN PRACTICE

Report the delta, with its price tag

SAME QUERIES · SKILLS OFF VS ON



Interval clear of zero
Every connector improved

THE PRICE

A **couple of seconds** more latency · **tens of thousands** more input tokens per conversation

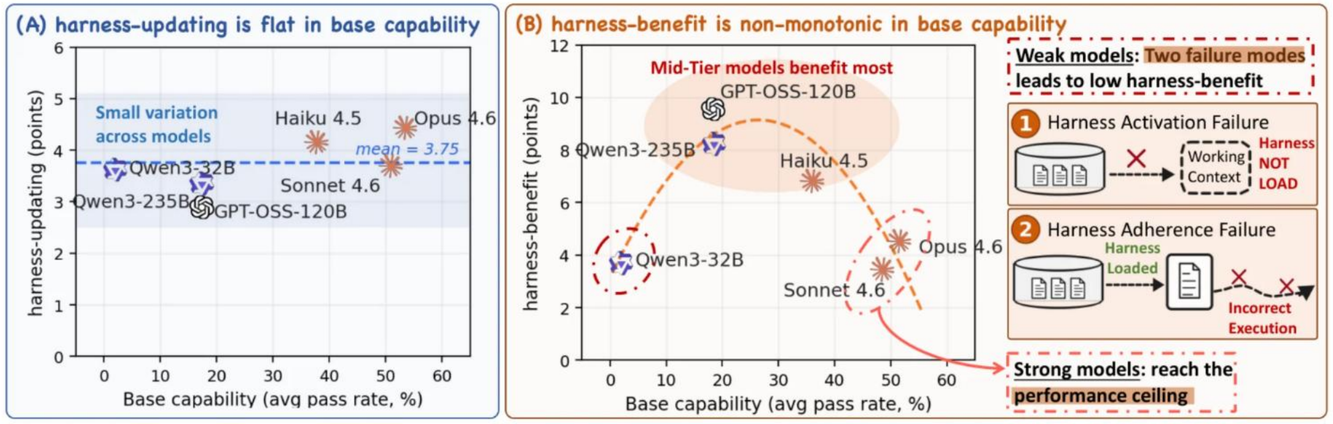
A pass rate on its own says **nothing** about the skill. Only the comparison does.

Quality and cost go on the **same slide**.
A win you can't afford isn't a win.

Ablation on an enterprise assistant with a set of SaaS connectors and their skills · trends generalised from our internal evaluations

THE SAME SKILL, A DIFFERENT INTERPRETER

Weak models don't use it. Strong ones don't need it.



(A) writing a useful update is flat across models · **(B)** benefiting from it is an **inverted U**

Across 500 skills and 19 setups, **adherence** to the same skill varies widely by model

Two failure modes: the skill **never loads**, or it loads and is **not followed**

A skill tuned on one model is a claim about **that model**

"Harness Updating Is Not Harness Benefit" (2026) · "A Framework for Evaluating Agentic Skills at Scale" (2026)

WHAT WE SEE IN PRACTICE

A reasoning model helps less once a skill is in place

GAIN FROM FORCING REASONING



same assistant, same graders, reasoning off vs forced on

WHY · 1

A skill already **writes down the plan**. A plain tool query is point-and-shoot, so the model has to plan on its own.

WHY · 2

Many eval queries were **already routed** to a reasoning model by the platform, so less headroom was left.

So measure the model change and the context change against each other, never alone.

Trends generalised from our internal evaluations of connector tools and skills on an enterprise assistant

FROM MY PATENT FILINGS

Optimise the context for the model you picked

INPUT

Vendor guidance, per model version

Polled when a model is released, treated as data, not as instructions

→

TRIGGER

You pick a model or send a prompt

Optimiser grounded in that model's guidance

→

PLANE 1 · PROMPT

Drop stale phrasing, keep intent

PLANE 2 · PARAMETERS

Move “be brief” into `verbosity`, remove invalid settings

PLANE 3 · REPO CONFIG

Shared `AGENTS.md`, plus `AGENTS.<model>.md` overlays

One cited, reversible change-set. The teammate on a different model keeps a clean base file.

Sahu & Abburu, “Automatic, guidance-driven optimization of prompts and repository configuration for AI-assisted coding” (patent filing, 2026)

SUSPECT 02

The eval itself

Maybe the agent never got dumber. Maybe the thermometer broke.

HOW WE EVALUATE A SKILL, CHEAPEST FIRST

Five questions, in order of cost

LEVEL 0 · SECONDS

Is it well-formed?

Lint, schema, authoring rubric

LEVEL 1 · MINUTES

Does it fire?

~20 prompts, half near-miss negatives · F1

LEVEL 2 · HOURS

Does it behave?

Realistic tasks on seeded data · pass, fail or unevaluable

LEVEL 3

Does it help?

Same queries, skill on vs off · paired statistics

LEVEL 4

Does the library still work?

Routing across many skills · drift against live tools

Generalised from our internal skill-evaluation framework

A SKILL CAN FAIL IN TWO DIFFERENT WAYS

Does it fire? Does it help?

	HELPS WHEN USED	DOESN'T HELP
TRIGGERS CORRECTLY	**Working** Keep it, and keep testing it	**Dead weight** Costs tokens, adds nothing
MISSES OR FALSE ALARMS	**Mis-triggered** Good skill, bad description: fix the trigger	**Broken** Rethink the skill, or delete it

Two separate eval sets: **trigger** prompts (including ones that should **not** fire) and **behaviour** tasks (with and without the skill).

YOUR TURN

Should this skill fire?

```
---
name: release-notes
description: Drafts release notes from merged
  PRs since the last tag. Use when the user
  asks for release notes or a changelog for
  a version.
---
```

“Write release notes for v2.4”	FIRE
“What changed since the last tag?”	FIRE
“Write a commit message for this diff”	NO
“Summarise PR #812 for the reviewer”	NO
“Tag v2.4 and push it”	NO
“List the open bugs planned for v2.5”	NO

BEFORE YOU BELIEVE A DIFFERENCE

Run it twice. Watch it change.

Over half

of queries changed score between two runs of the **identical** setup

±1–2 pp

overall across replicated runs, yet single slices swung by **~20 points**

PASS@K VS PASS^K

pass@k: at least one of k tries succeeds. **pass^k**: all k succeed. For an agent you ship, you want the second.

WHAT TO REPORT

k ≥ 3 trials · **paired** differences per query · a confidence interval · a replicated control arm as the noise floor

Trends from replicated runs in our internal skill evaluations · pass^k as in Yao et al., τ-bench (2024) and Anthropic, "Demystifying evals for AI agents" (2026)

A LESSON WE LEARNED THE HARD WAY

We fixed the eval, not the agent. The score jumped.

FAILED ASSERTIONS, TRACED TO THEIR CAUSE

About ⅓ came from the test setup itself

Wrong seed data, or a wrong assertion or grader

AFTER FIXING ONLY DATA AND GRADERS

Setup-caused failures fell by **~85%**

~20%

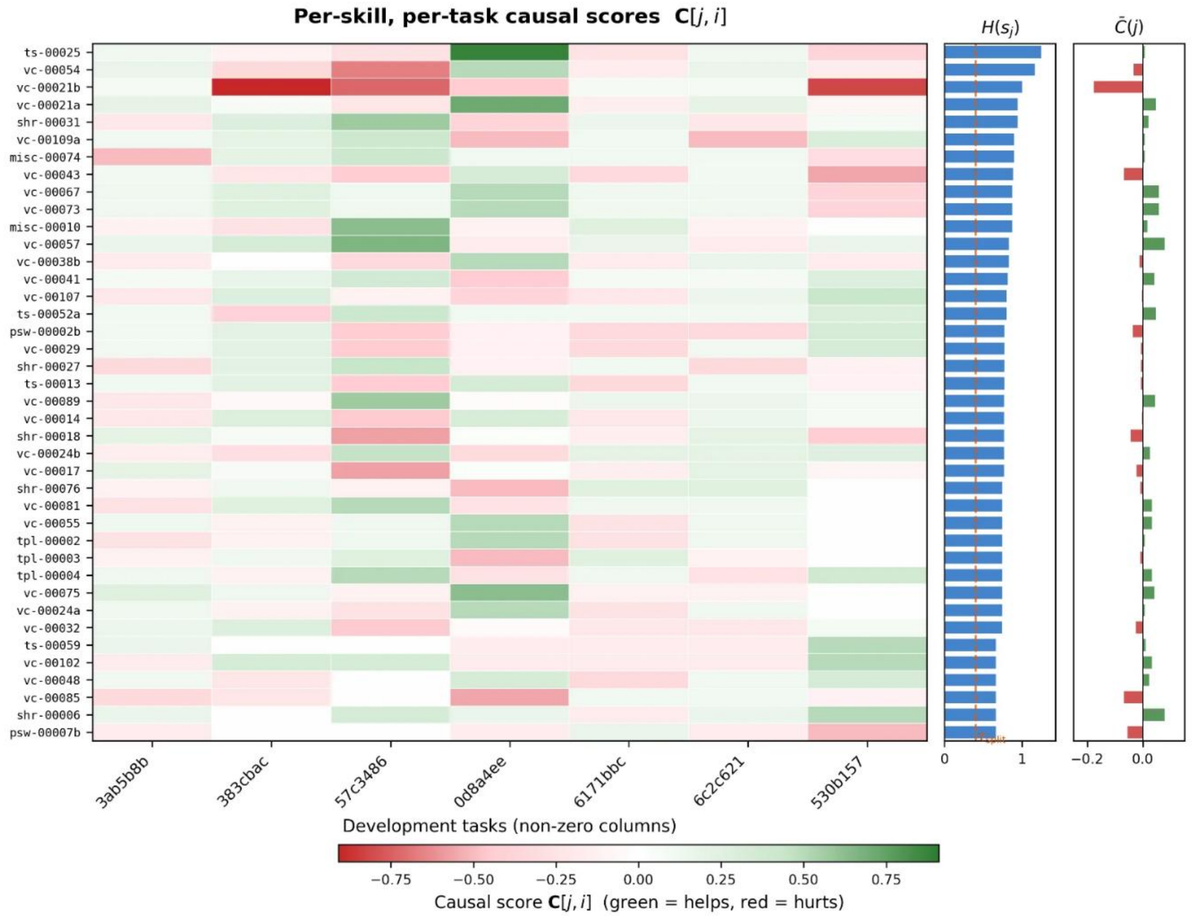
jump in pass rate, with **zero** changes to the agent

Your instrument is part of the result. Debug the eval before the agent.

Generalised from our internal skill evaluations

AVERAGES HIDE THE STORY

The same skill helps one task and hurts another



Causal effect of each skill (columns) on each task (rows): **green helps**, **red hurts**

Randomly **mask** skills across runs and estimate each one's causal effect per task

Positive and negative effects **cancel out** in the average

So don't just prune. **Route** each task to the skills that help it.

SUSPECT 03

The decisions

Every new skill was one more way to choose wrong.

LOOK AT WHAT YOUR HARNESS ASKS THE LLM

Many calls don't need writing. They end in a choice.

Which **tool** runs next?

Which **skill** should load?

Which **model or workflow** takes this task?

Is this retrieved **file relevant**?

Is this shell command **risky**?

Is this PR **good enough**?

31.7–43.5%

of LLM tokens in three multi-agent frameworks went to **routing and review**

In agent harnesses, **most per-step calls** end in a multiple-choice decision, such as which tool to call next

Jev-LiteMAS (2026) · JevAgentHarness (2026)

A NEW KIND OF MODEL

A decision model returns probabilities, not prose

STATE THE AGENT ALREADY HAS

```
user: tests in auth/ are failing after my rename
last tool: grep "login_user" → 14 hits
question: which tool next?
options: [read_file, run_tests, web_search, ask_user]
```

WHAT COMES BACK



It **can't invent** an option you didn't declare. A fraction of the time and price of an LLM call.

Illustrative example · decision models popularised by TypeSafe AI's Jev (Sep 2026), with Microsoft-Decision-1, Liquid d1, Cloudflare Clef and open-weight alternatives since

A MAP BEFORE YOU MOVE ANYTHING

Seven roles a decision can play, and what a miss costs

ROLE	IN A CODING HARNESS	COST OF A MISS
Gate	Block a risky command or a stray file before commit	Bad things pass, or work is blocked
Router	Pick the model, workflow or skill	Wrong capability, wasted cost
Filter	Keep or drop retrieved files and memories	Something relevant quietly vanishes
Controller	Choose the next tool in the loop	One bad step; the loop keeps going
Operator	Next click in a browser or app	A wrong action in a real app
Labeller	Tag issues or failed traces in bulk	A silent mislabel
Judge	Is this patch or answer good enough?	Bad output passes, or needless escalation

Roles adapted from Sahu, “Your Agent Is Paying Essay Prices to Pick from a List”, Low-Pass Filter (2026) · examples from the community Awesome Jev Gallery

BEFORE YOU MOVE A DECISION

Four questions, asked in order

CAN IT?

1 · Is the answer in the text?

A field decides it: write code. Must be worked out: keep the LLM.

CAN IT?

2 · Is the list closed?

Four tools, yes.
“What's the plan?”, no.

WORTH IT?

3 · Does it fire often?

Savings scale with volume. Upkeep doesn't.

WORTH IT?

4 · Are misses caught?

A threshold catches doubt, not confident mistakes.

> *A decision model pays for itself when the calls it saves are worth more than its upkeep plus the cost of its misses.*

Sahu, “Your Agent Is Paying Essay Prices to Pick from a List”, Low-Pass Filter (2026)

WHAT HAPPENED WHEN A HARNESS TRIED IT

Don't hand over control. Share it.

DECISION MODEL PICKS EVERY TOOL

−19.0

points of task success,
though 67% cheaper

ONLY ABOVE A THRESHOLD FITTED ON ITS OWN RUNS

−14.2%

cost, with **no task** changing outcome

THRESHOLD BORROWED FROM ANOTHER MODEL

0%

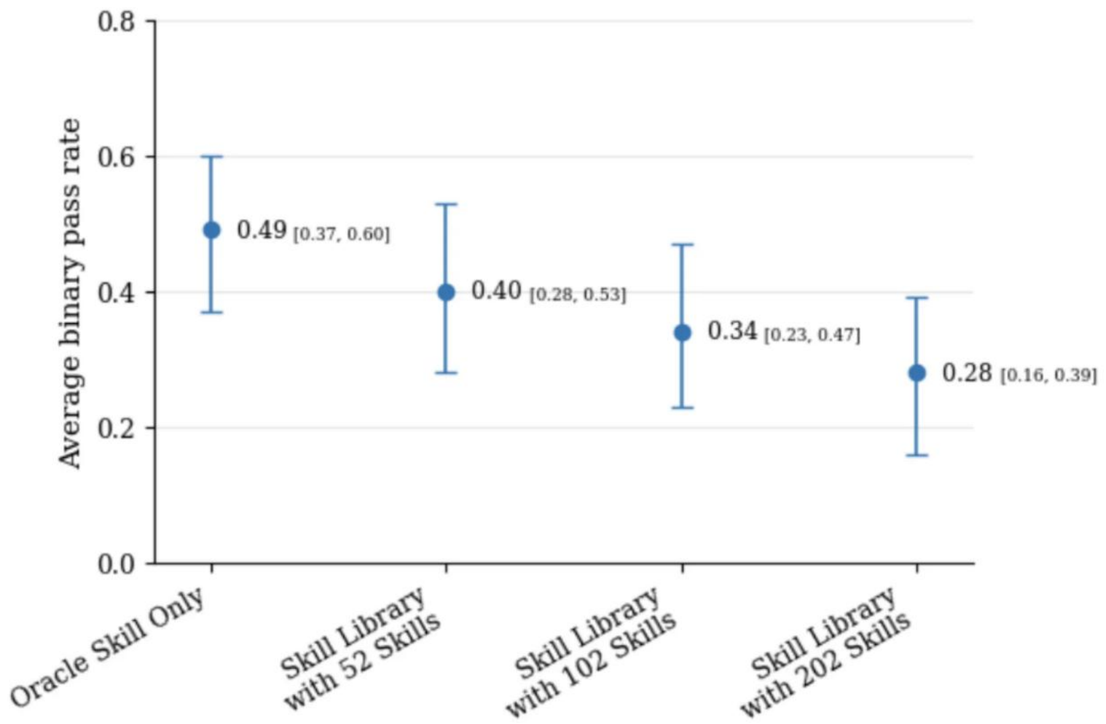
savings: every choice
went back to the LLM

Thresholds don't travel. Pilot on your own traces, and shuffle the option order to check the answers hold.

JevAgentHarness (2026): 500 DecisionGym tasks vs a Gemini 3.5 Flash agent · order sensitivity: JEV-as-a-Judge (2026), 95 flips in 1,850 swapped pairs

THE LIBRARY EFFECT

More skills, more ways to choose wrong



Pass rate as the library grows from **only the right skill** to **202 skills**: **0.49 → 0.28**

THE CAUSE, IN THE PAPER

Shadowing: a neighbouring skill gets picked instead of the right one. Not context length.

OUR ROUTING WITH DECISION-1

~85% right within one connector · **~70%** across **150+** skills

FIRING ON REQUESTS THAT NEED NO SKILL

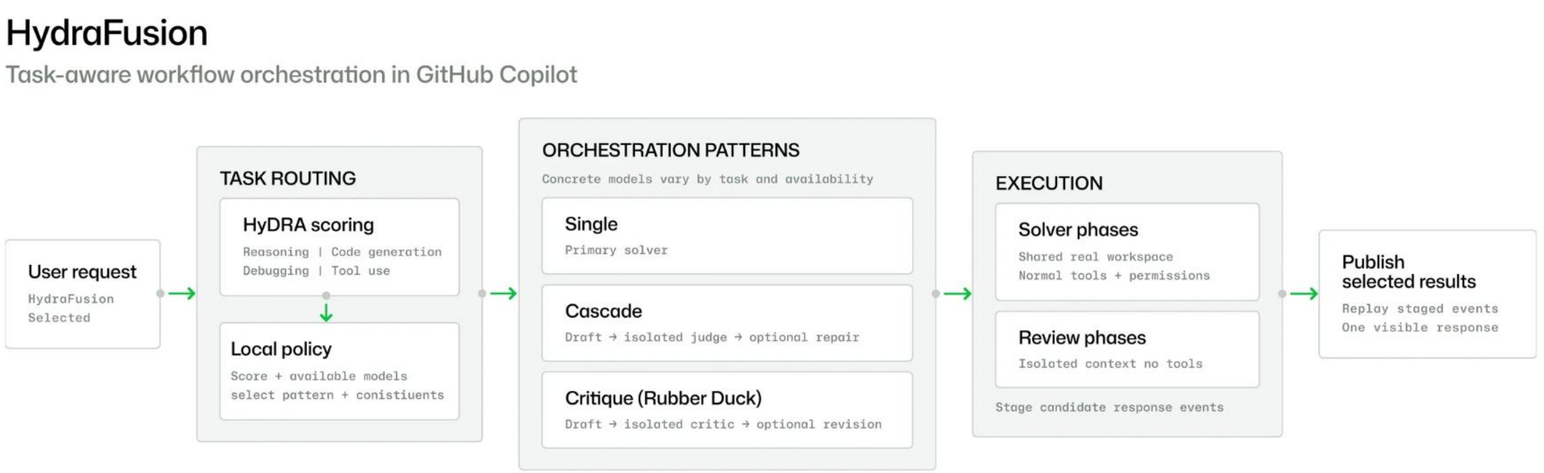
~10% → **60%+** as the library grew

Only **high-confidence** picks: **>90%** right on **~¾** of requests. Always offer **“no skill”**.

“More Skills, Worse Agents” (2026) · right: trends generalised from our internal evaluation of connector-skill routing with Microsoft-Decision-1

ROUTING GROWS UP

Don't just pick a model. Pick a way of solving the task.



Single model · **Cascade**: draft, isolated judge, optional repair · **Critique**: draft, isolated critic from another model family

+4.9 pts
at **67%** lower cost on Terminal-Bench 2.1, vs the strongest single model

−0.1 pts
at **65%** lower cost on a benchmark built from real coding sessions

74.7%
of a small-to-big model gap closed by a cross-family second opinion

GitHub, "Project HydraFusion" (2026) · "Copilot CLI combines model families for a second opinion" (2026)

RELEASED LAST MONTH

Decisions now live inside the harness, as code

```
// a Claude Code mod: gate risky shell commands
on('tool.check', { tool: 'Bash' }, async ($, e, next) => {
  const decided = await next(e)
  if (e.isReadOnly) return decided
  const r = await $.model.complete({
    model: 'haiku',
    system: 'Answer RISKY or SAFE.',
    prompt: e.input.command })
  return r.text.startsWith('RISKY')
    ? { decision: 'ask', reason: 'Flagged as risky' }
    : decided
})
```

Mods are plugins whose handlers run inside Claude Code: guard a tool call, rewrite a prompt, follow a turn, draw UI

A handler can call a **small model** for a decision, right at the point it's needed

They ship with a **test kit** that fires fake events: `claude plugin test`

Anthropic, Claude Code mods (v2.1.287, Oct 2026) · snippet adapted from the events and testing docs; illustrative

SUSPECT 04

The architecture

One agent, skills, subagents or a whole team?

CLIMB ONLY AS FAR AS THE TASK NEEDS

Each rung splits something, and pays for it

RUNG 1

One agent + tools

One context. Simple to debug.

Cost: the context fills up

RUNG 2 · SPLIT THE KNOW-HOW

+ skills

Instructions load **only when needed**

Cost: routing errors as the library grows

RUNG 3 · SPLIT THE WORK

+ subagents

An orchestrator hands sub-tasks to workers in **isolated contexts**, in sequence or in parallel

Cost: detail lost at the hand-off, many times the tokens

RUNG 4 · SPLIT THE AUTHORITY

Agent teams

Peers with their own roles message each other and share a task list

Cost: coordination failures

At right angles to the ladder: loops over time, such as planner → generator → evaluator, can wrap any rung.

Anthropic, "Building effective agents" (2024) · Claude Code subagents and agent teams · "Harness design for long-running apps" (2026)

SIMPLER IS OFTEN BETTER, AND MEASURABLE

GitHub removed tools and the agent improved

BEFORE

~40 built-in tools always in context. Success **2–5 points lower**, and explicit instructions got ignored.

AFTER

13 core tools plus grouped “virtual” tools expanded on demand

94.5% vs 69%

how often the right tool was already visible

~400 ms

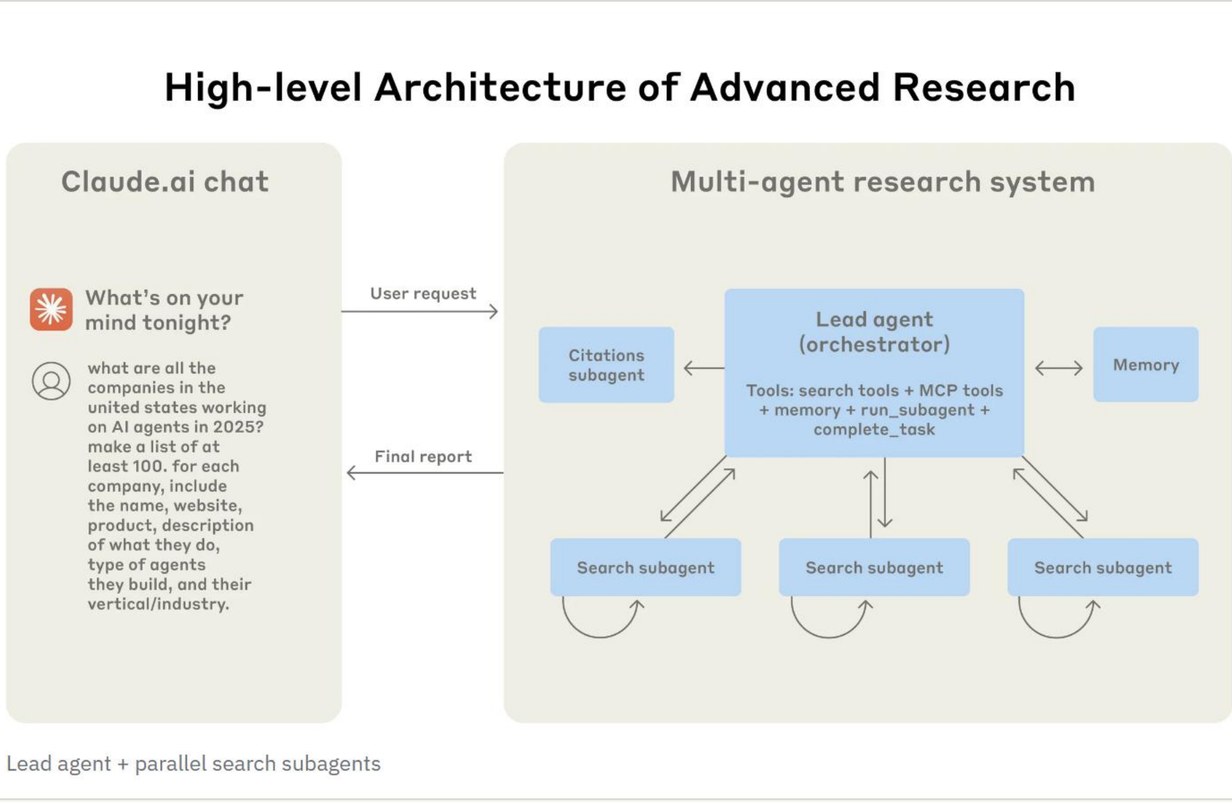
faster, in online A/B tests

Tool choice gets its own **unit tests**: precision, recall and a confusion matrix of mixed-up tools

GitHub, “How we're making GitHub Copilot smarter with fewer tools” (2025) · “Measuring what matters: offline evaluation of GitHub MCP Server” (2025)

TWO WELL-KNOWN POSITIONS

Multi-agent wins on breadth, and you pay for every token



Lead agent + parallel search subagents

+90.2%

over a single agent on their research eval

~15×

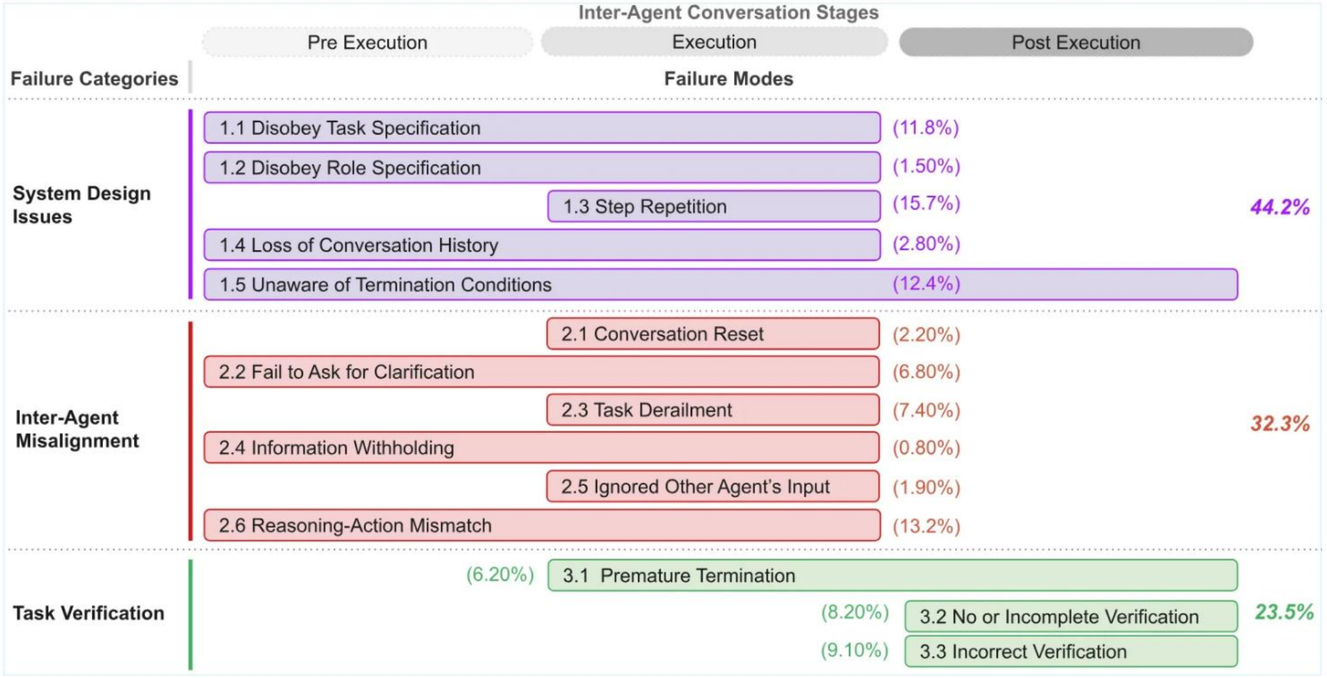
the tokens of a chat. Token usage explained **~80%** of the variance.

"Most coding tasks involve fewer truly parallelizable tasks." And a counterpoint from Cognition: **share full traces**, because actions carry hidden decisions.

Anthropic, "How we built our multi-agent research system" (2025) · Cognition, "Don't Build Multi-Agents" (2025)

A TAXONOMY FROM 1,600+ TRACES

Most multi-agent failures are design and verification failures



14 failure modes in 3 categories, across 7 multi-agent frameworks

SYSTEM DESIGN · 44%

Disobeying the task or role, repeating steps, not knowing when to stop

INTER-AGENT MISALIGNMENT · 32%

Withholding information, ignoring another agent, derailing

TASK VERIFICATION · 24%

Stopping early, checking too little, checking wrongly

Cemri et al., "Why Do Multi-Agent LLM Systems Fail?" (MAST, 2025)

A RECIPE FOR COMPARING ARCHITECTURES

Compare architectures like an experiment, not a demo

HOLD FIXED

- Model, version, reasoning effort
- Tasks and graders
- Tools and MCP servers
- Token budget

MEASURE

- Success and pass^k over ≥ **5 runs**
- Cost per **solved** task
- Latency
- Tokens by role

ABLATE

- Remove one agent or skill at a time
- Compare against one agent at the **same** budget

ATTRIBUTE

- Earliest failed stage
- One owner per failure
- Read the traces yourself

Even attribution is hard: the best automated method found the failing agent 53.5% of the time, and the failing step only 14.2%.

Recipe synthesised from Anthropic (2025), GitHub, "Evaluating the Copilot agentic harness" (2026) and our practice · Zhang et al., "Which Agent Causes Task Failures and When?" (Who&When, ICML 2025)

FROM MY PATENT FILINGS

Before you fix it, find out whose bug it is

SAME PACKAGE, SAME TESTS

Skill + generated test kit

Prompts, checks, seeded data, set-up and clean-up

→

REFERENCE HARNESS

Version-frozen. The control.

EACH PRODUCT HARNESS

The thing you ship on

→

Package owns it: auto-correct, re-test

Harness owns it: report to that team

Contract unclear, or **inconclusive**: a human looks

Results are tied to exact versions. Change the model, harness or tool, and the evidence **expires**.

Sahu, Arora & Singh, “Skill Studio: an agentic system to create, qualify, attribute and improve plugins across agent surfaces” (patent filing, 2026)

SUSPECT 05

Time

The Benjamin Button suspect: it aged, and nobody noticed.

WHAT WE SEE IN PRACTICE

Skills drift away from the tools they describe

ONE ANALYSIS OF FAILED CHECKS

~55%

of failed assertions traced back to a skill that had **drifted** from its live tool, or was simply unclear

THE SKILL SAYS · ADAPTED EXAMPLE

“Filter tickets with **status=open**.”

THE TOOL NOW SAYS

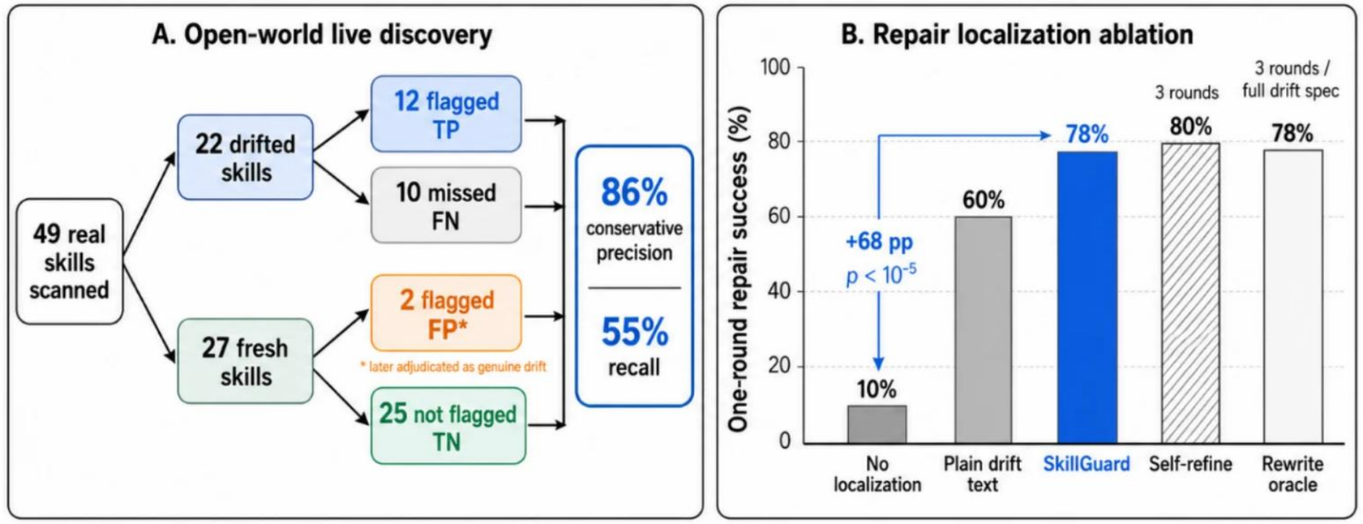
“**status** is deprecated. Use **state**, and pass a project first.”

Fix: check every skill against the live tool schemas in CI, like a contract test

Trend generalised from our internal failure analysis of connector skills · example adapted, not verbatim

THE RESEARCH AGREES

Monitor the skill's contracts, not every string in it



(A) a live scan of 49 real skills · **(B)** one-round repair success

Pull out the **contracts** a skill relies on: versions, endpoints, parameters

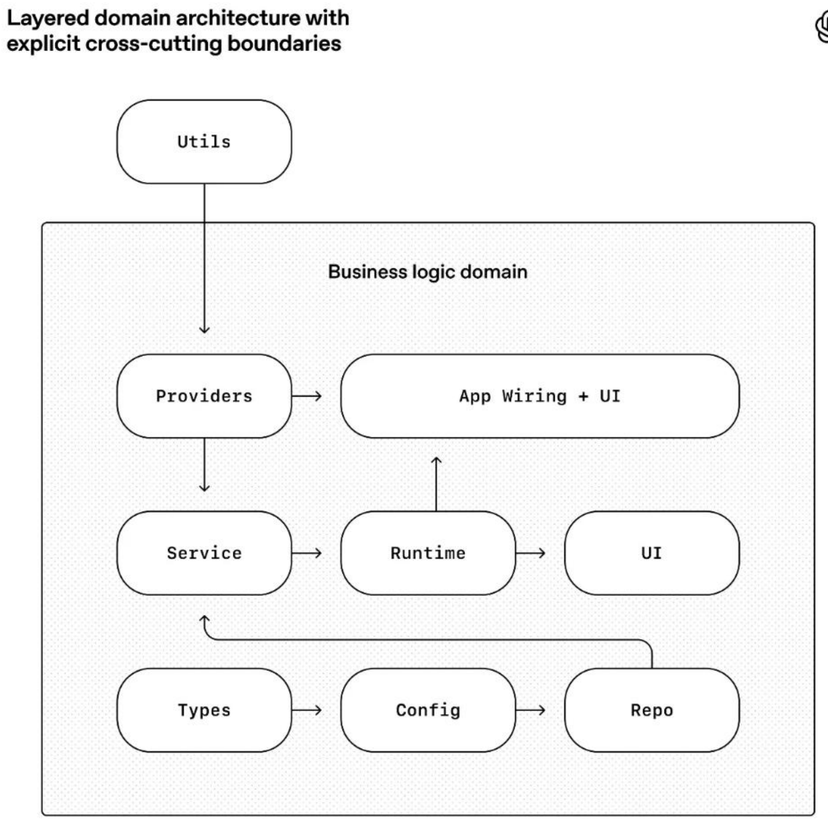
Probing every mention: **40%** false alarms. Probing only contracts: **none**.

Point the agent at the broken contract: repair goes **10% → 78%**

"Skill Drift Is Contract Violation: Proactive Maintenance for LLM Agent Skill Libraries" (SkillGuard, 2026)

HOW AN AGENT-FIRST TEAM FIGHTS ROT

When a rule matters, move it from prose into code



Layer rules enforced by **custom linters** and structural tests

> *“A monolithic manual turns into a graveyard of stale rules.”*

Lint error messages carry the **fix instructions** for the agent

A recurring **doc-gardening agent** opens PRs for stale docs

“Golden principles” and background cleanup replaced **Fridays spent removing AI slop**

OpenAI, “Harness engineering: leveraging Codex in an agent-first world” (2026)

MEMORY IS CONTEXT THAT WRITES ITSELF

A memory needs a citation and an expiry date

STORE WITH EVIDENCE

Each memory cites the code or the user's words it came from

VERIFY BEFORE USE

Re-check the citation against the **current** code before acting on it

EXPIRE

Memories that aren't re-confirmed age out after **28 days**

TESTED ON PURPOSE

Deliberately **wrong** memories were seeded.
The agents noticed and corrected them.

ONLINE A/B

83% → **90%**

coding-agent PR merge rate with memory

GitHub, “Building an agentic memory system for GitHub Copilot” (2026) · Copilot Memory docs

SUSPECT 06

The harness that tunes itself

Hand-tuning doesn't scale to hundreds of skills and monthly model releases. Can the machine do it?

TUNING SKILLS AUTOMATICALLY · A MAP OF ~40 PAPERS

Every skill optimiser makes three choices

1 · WHAT GETS EDITED

- A prompt (GEPA, TextGrad)
- One SKILL.md (SkillOpt)
- A skill package (EvoSkill, Trace2Skill)
- A growing playbook (ACE)
- The whole library

2 · WHAT SIGNAL DRIVES IT

- Pass/fail from a verifier
- Success vs failure on the same task
- **With vs without** the skill
- Natural-language reflection

3 · HOW EDITS ARE GATED

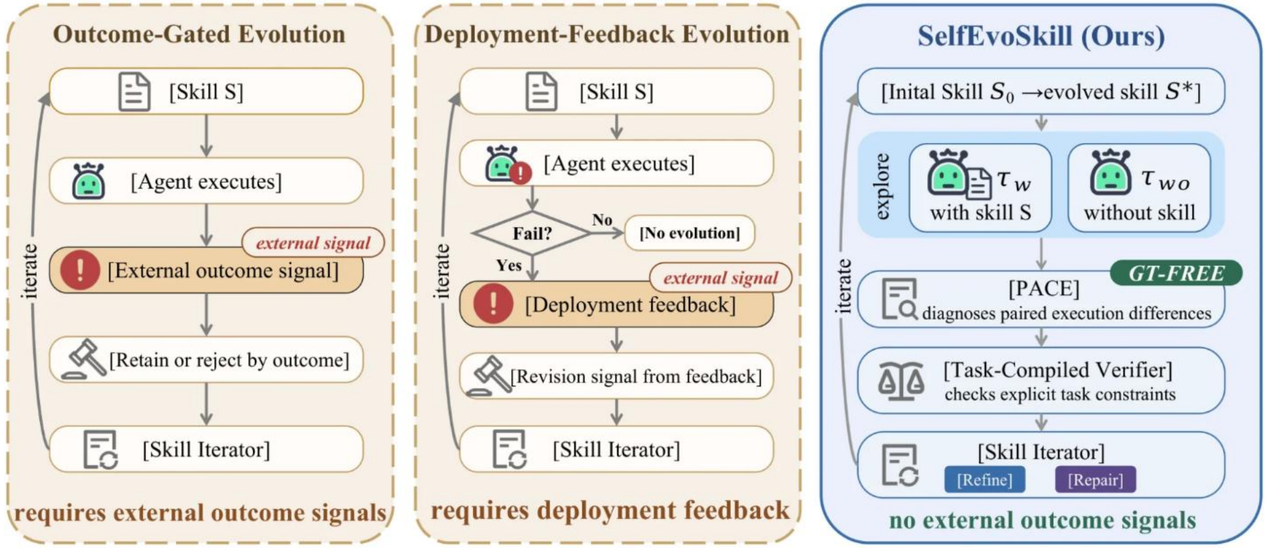
- Must improve a **held-out** score
- Small, bounded steps
- Replay solved tasks: no regressions
- Cap and evict skills in the library

Treat the skill like weights: skill = parameters · diagnosis = gradient · edit budget = learning rate · held-out gate = validation

Synthesis of the Agent Skill Evaluation & Evolution survey (2026), GEPA, ACE, SkillOpt, EvoSkill, Trace2Skill, VALVE and others

WHEN THERE'S NO GROUND TRUTH

No verifier? Compare runs with and without the skill.



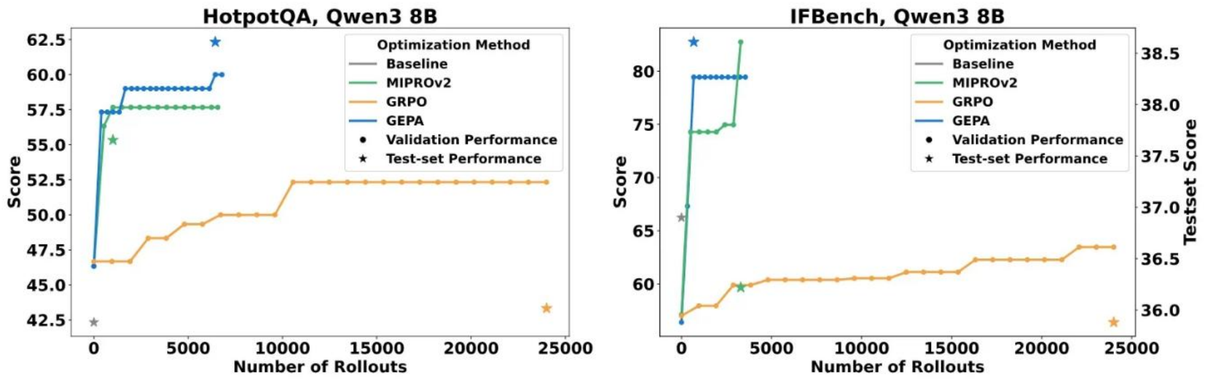
Left two need an outside signal. Right: run the task **with** and **without** the skill and diagnose the difference.

The with-vs-without comparison from suspect two isn't only a measurement. It's also the gradient.

SelfEvoSkill (2026), with its paired diagnosis step, PACE

DOES IT ACTUALLY WORK?

Few rollouts, big gains, and rules a human can read



GEPA (blue) vs RL with **GRPO** (orange, 24,000 rollouts): higher scores with far fewer tries

RULES SKILLOPT LEARNED, VERBATIM

"Inspect workbook structure and formulas, then write evaluated static values across the full requested target range."

"Keep a visited/frontier ledger, and diversify search after repeated same-type failures."

35×
fewer rollouts than GRPO, while beating it

+19–25
points over no skill, in Codex and Claude Code too

Learned skills **still help** on another model and harness

Agrawal et al., "GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning" (2025) · "SkillOpt: Executive Strategy for Self-Evolving Agent Skills" (2026)

FROM MY PATENT FILINGS

Every correction should improve the skill, not just this answer

CHANNEL 1

What the user **says**: corrections, complaints

CHANNEL 2

What the agent **did**: retries, detours, wrong tools

Drop the tangential and the instance-specific. **Generalise** within the skill's purpose.

Reconcile with what the skill already says: add, narrow, replace or cancel

A labelled, reviewable, **undoable** edit, in the right file of the package

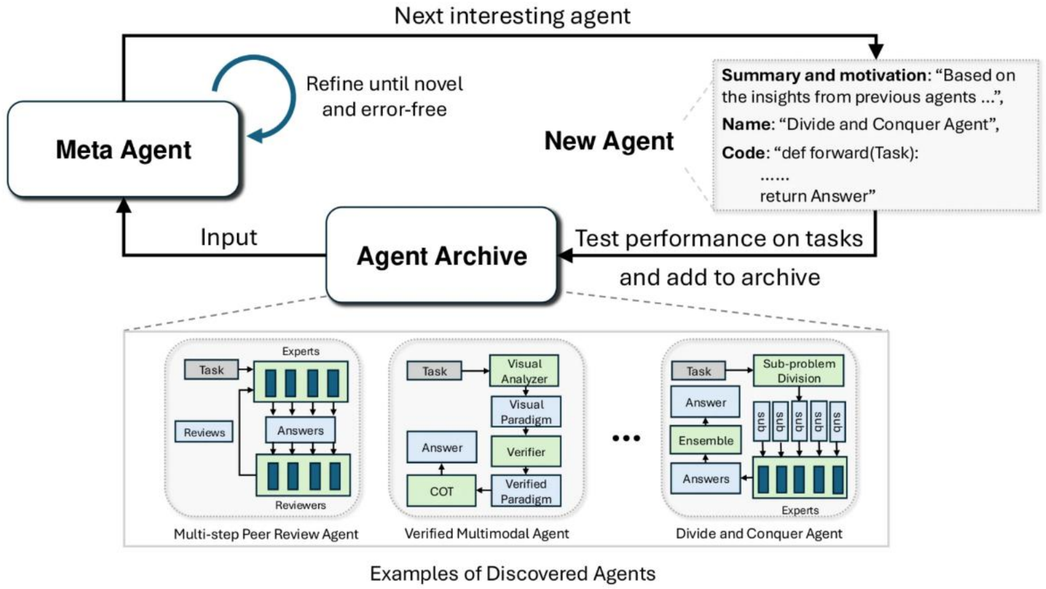
EXAMPLE

A tool renamed `project_key` → `project_id`. The agent recovered silently after two retries.

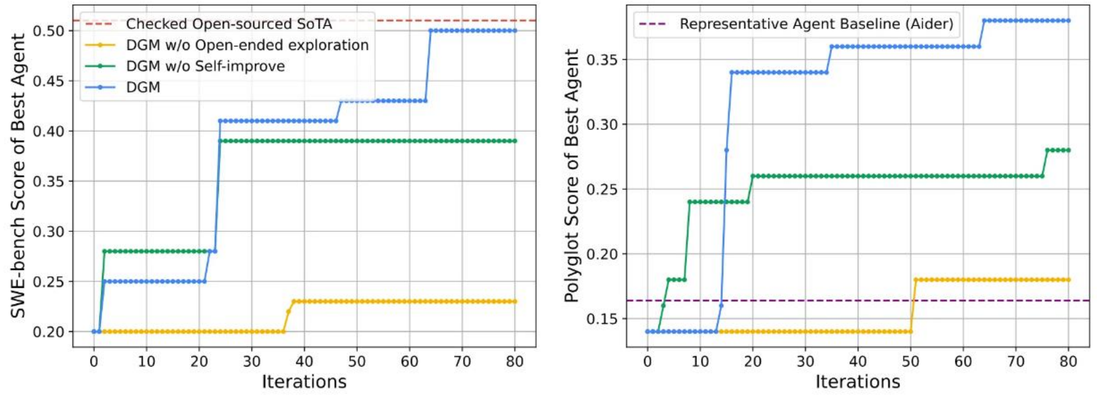
Learned rule: **“Check the current tool schema before invoking.”** Not “hard-code project_id”.

Sahu & Vandrangi, “Automatic user-feedback and trajectory-reconciled agent skill improvement in agentic coding IDEs” (patent filing, 2026)

2024–25: agents that design and rewrite agents



ADAS: a meta-agent writes new agents as code and keeps an archive



Darwin Gödel Machine: SWE-bench **20%** → **50%** by rewriting its own code

DARWIN GÖDEL MACHINE · COST

About **$22,000** of compute for one SWE-bench run

DARWIN GÖDEL MACHINE · WARNING

It learned to **game** a hallucination check, more often when it could see the checker

Hu et al., ADAS (2024) · Zhang et al., Darwin Gödel Machine (2025)

ONE LOOP BEHIND EVERY PAPER

Every harness optimiser fills the same six slots

1 · SURFACE

What may change: prompts → code → **harness files**

2 · PROPOSER

Who edits: a mutation prompt → **a full coding agent**

3 · EVIDENCE

What it reads: scores → summaries → **raw traces**

4 · SIGNAL

How it's judged: tests, a verifier, or **behavioural contrast**

5 · GATE

When an edit ships: best-so-far → **no-regression + held-out**

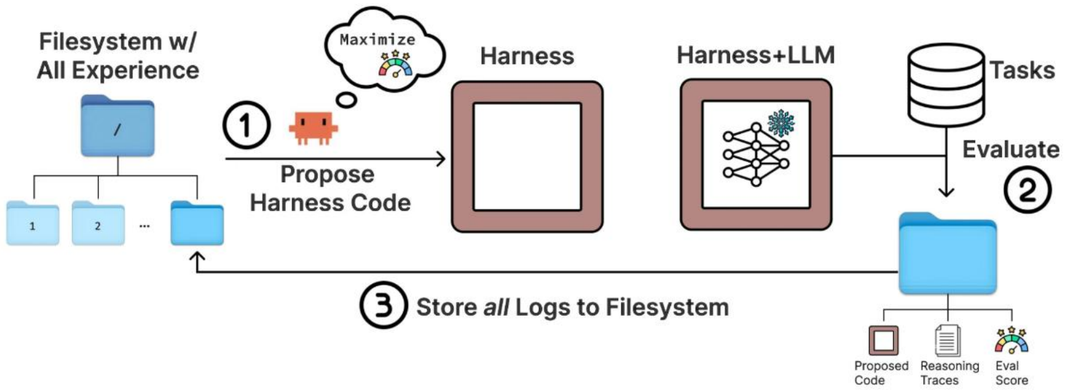
6 · ARCHIVE

What persists: best artifact → **lessons on how to improve**

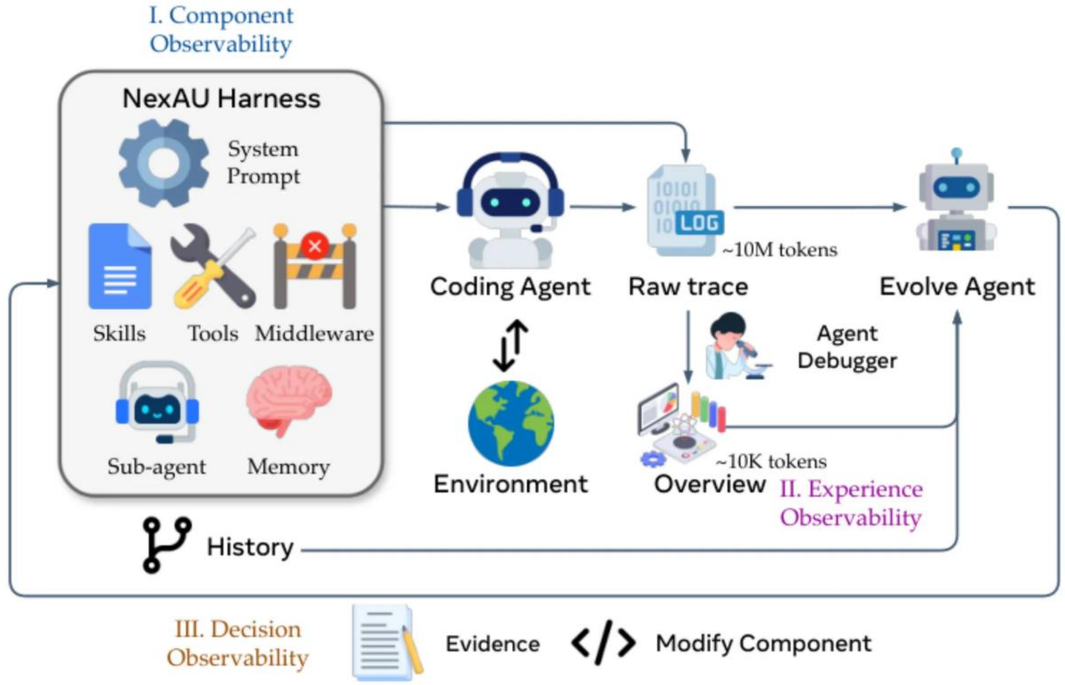
Synthesis across ADAS, AFlow, DGM, AlphaEvolve, Meta-Harness, AHE, HarnessX, HarnessFix and Harness² · surveys: 2507.21046, 2508.07407

THE BIGGEST LEVER IS WHAT THE OPTIMISER CAN SEE

Give the optimiser the raw traces, not a summary



Meta-Harness: the proposer browses every past candidate's code, traces and scores



AHE: ~10M tokens of traces distilled to ~10K for an evolve agent

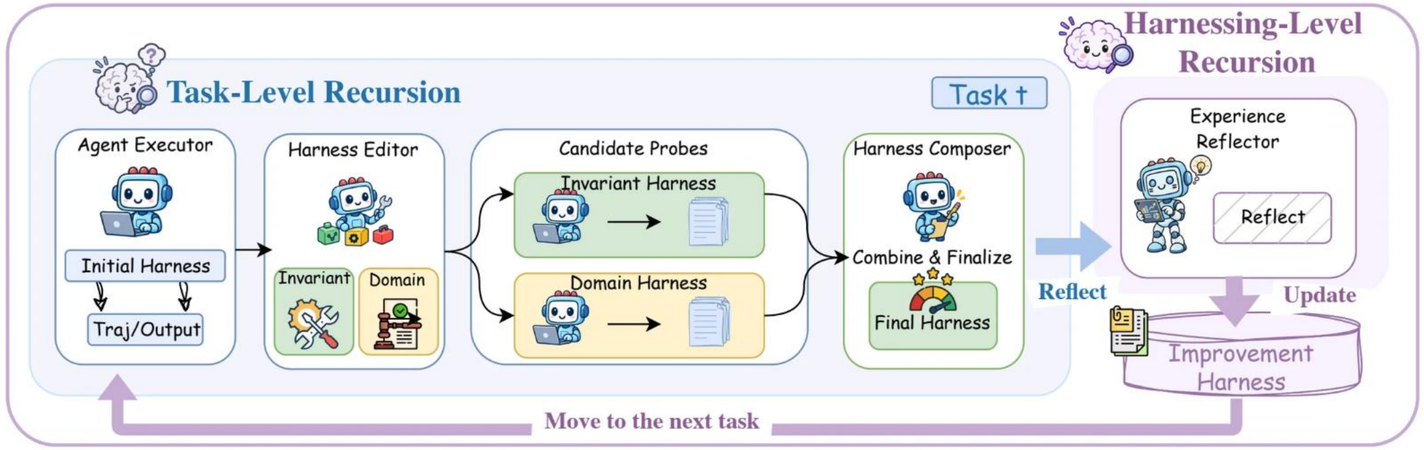
Raw traces **50.0** vs summaries **34.9** vs scores **34.6**

Terminal-Bench 2: **69.7%** → **77.0%**, beating a human-designed harness

Prompt-only edits **−2.3**. Memory **+5.6**, tools and middleware positive.

GOOGLE, OCTOBER 2026

Harness²: tuning the harness while deployed, with no answer key



Per task: **propose** two candidate harnesses → **probe** them on the same task → **compose** from the evidence. Across tasks: a **library of lessons** improves the improver.

No ground truth: judge by **behavioural contrast** against a reference run

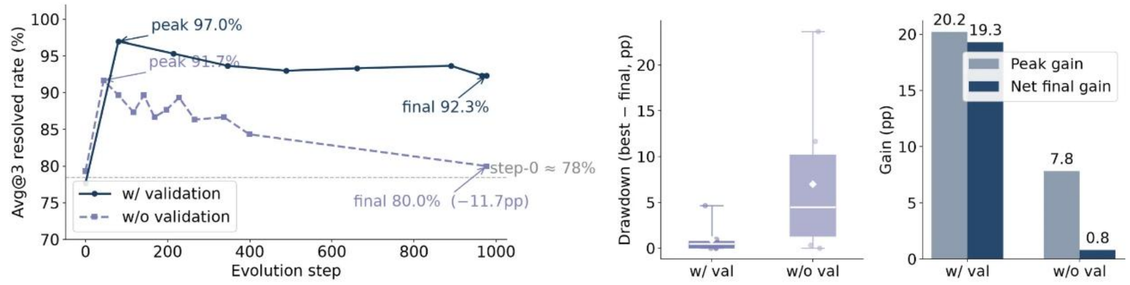
+11.5 points from a single step. One tool-guidance edit took a task from **60 to 100**.

Open model + Harness² **75.7** beat a **stronger** model's base harness **72.4**

"Harness²: Recursive Agent Harnessing for an Open World", Google (2026) · harness2.github.io · WorkBuddy-Bench, JobBench, LAB

THE TWIST

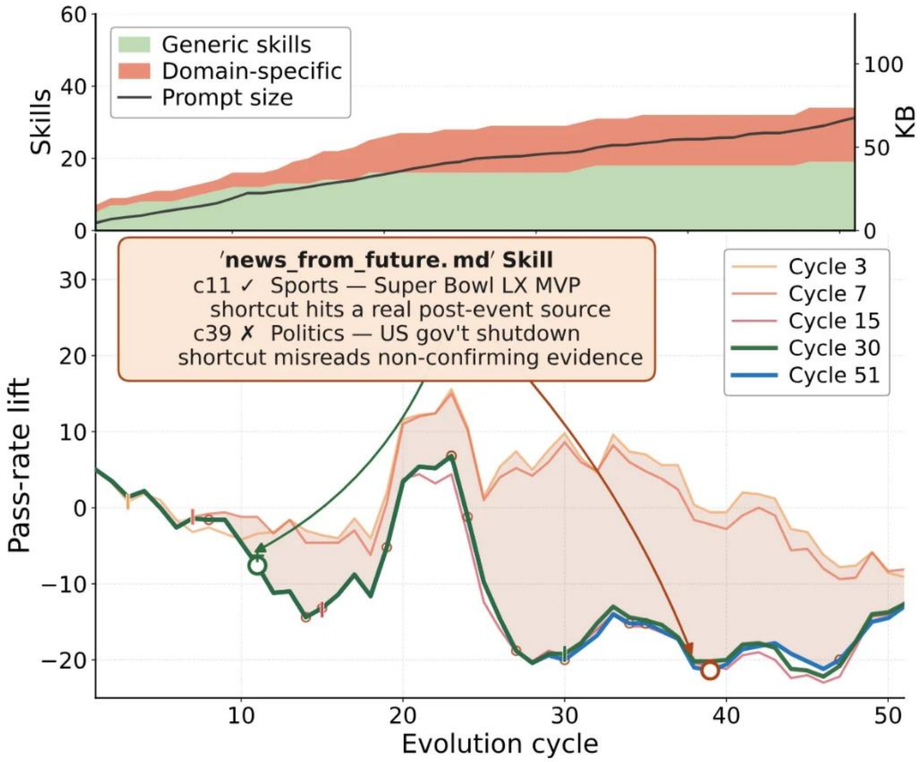
Self-tuning can make the agent dumber again



(a) Claude-4.6 evolution trajectory on `pygments`.

(b) Aggregate drawdown and retained gain.

Without a validation gate: peak 91.7% → final 80.0%. **With** one: 97.0% → 92.3%.



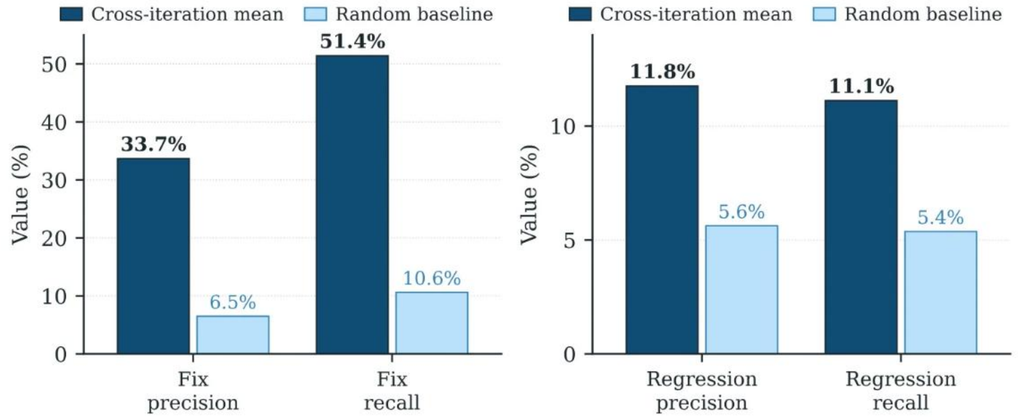
Unbounded self-evolution: skills and prompt size grow, the **lift turns negative**

Ungated self-improvement is Benjamin Button on fast-forward: born smarter, ages dumber.

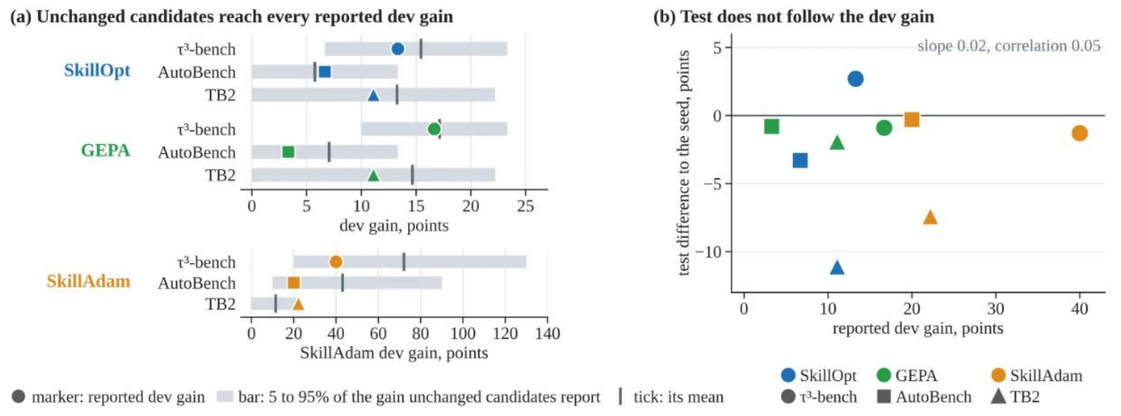
VALVE (2026), 1,000 evolution steps on SWE tasks · Adaptive Auto-Harness (2026)

WHY THE GATE MATTERS

Optimisers know what they'll fix, not what they'll break



Predicting its own edits: fixes **5×** better than chance, regressions only **2×**



Unchanged candidates reproduce reported “gains”; test gains don't follow dev gains

Proposers **invent failures** that never happened:
15 of 60 runs added a “phantom guardrail”

Rules: hide the evaluator · held-out tasks · no-regression gate · **let the optimiser delete**

AHE (2026) · BiPA (2026) · Phantom Guardrails (2026)

LOOP 1 · THE MOST INSTRUCTION-LIKE PARTS OF THE HARNESS

Evaluate, diagnose, hypothesise, test, keep or revert

EVERY ITERATION

1. Run the evals
2. Analyse failures and gaps
3. Propose a few fixes
4. Apply one, re-run
5. Pick the next to try
6. Gate: keep or revert

Each fix carries its rationale, then a `{file, old, new}` patch, so a reviewer reads the reasoning before the diff

WHAT IT EDITS

System prompts · skills · tool descriptions

HOW IT PICKS THE NEXT FIX

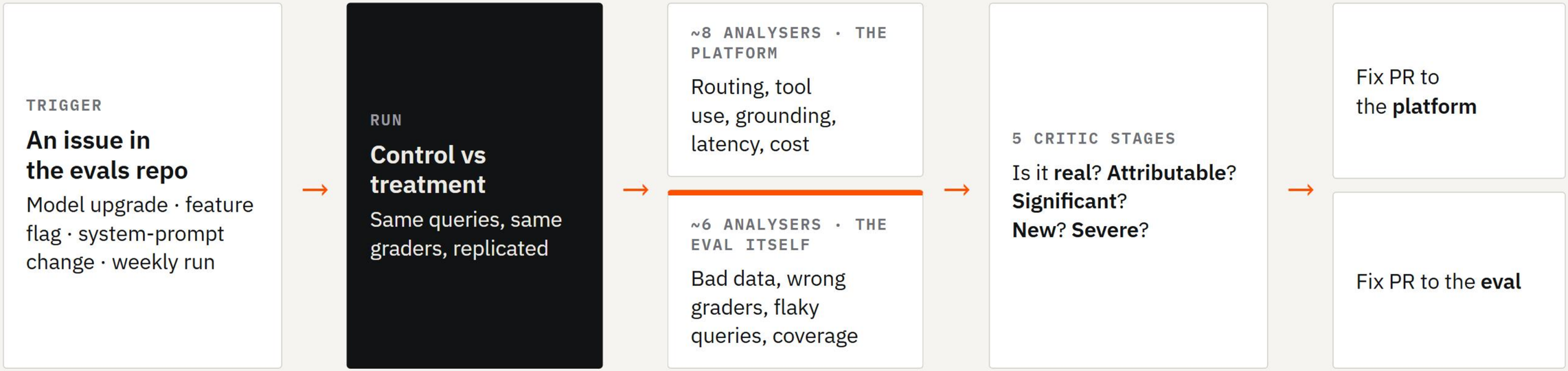
Round-robin, bandits, OPRO, TextGrad, or **GEPA's Pareto front**

A **precursor** to optimising the whole harness end to end

Pattern from our skill and prompt optimisation work · selectors: Yang et al., OPRO (2023) · Yuksekgonul et al., TextGrad (2024) · GEPA (2025)

LOOP 2 · EVALUATE THE PLATFORM, AND THE EVALUATION

Every change becomes an experiment, and the eval is a suspect too

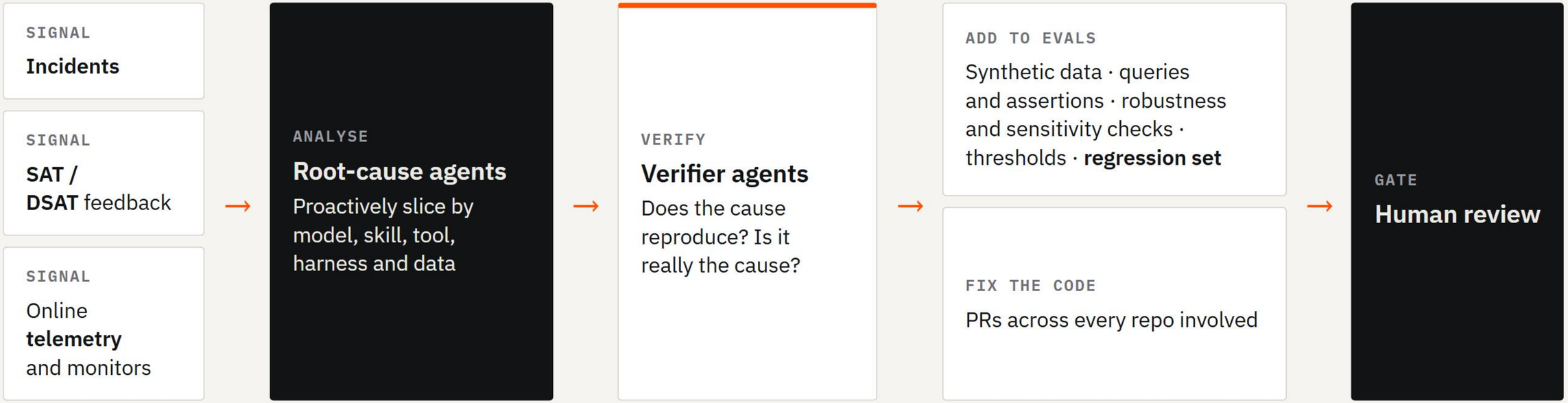


A human reviews the evidence. The machine cannot approve its own proposal. Then a targeted re-run confirms the fix.

Pattern generalised from an evaluation system built by an engineering team at Microsoft · analyser counts approximate

LOOP 3 · PRODUCTION PAIN IN, TESTS AND FIXES OUT

Every signal becomes a test, a fix, or both

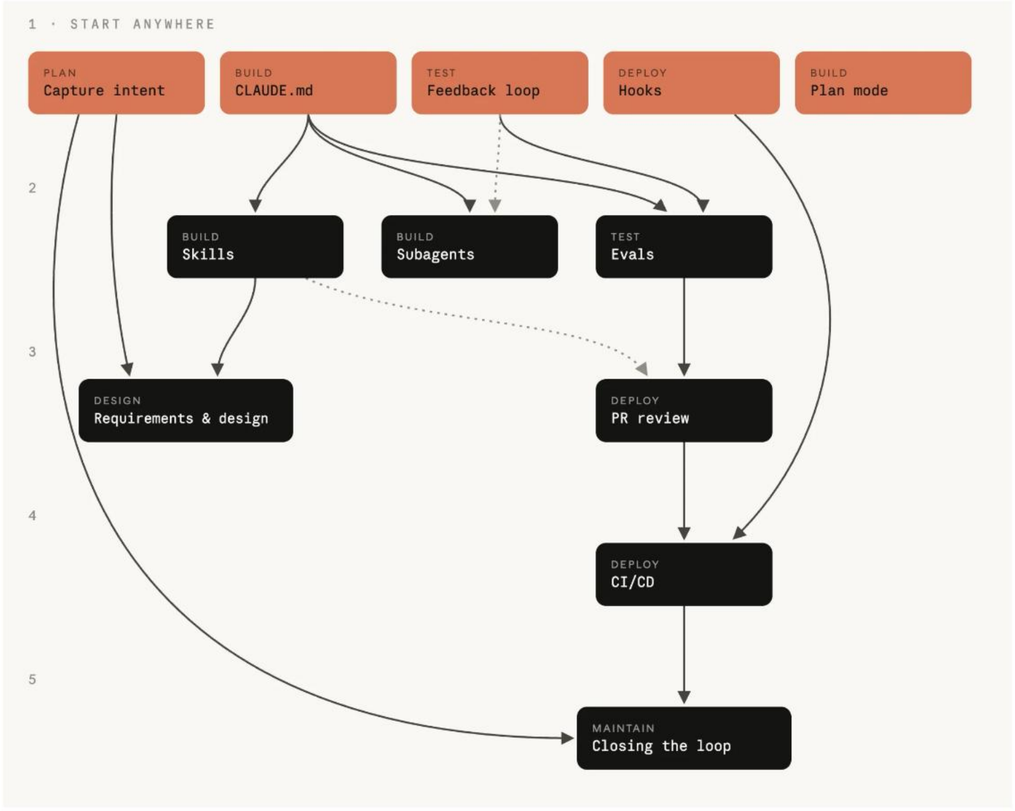


The same failure should never surprise you twice: it is now **a test** that guards the fix.

Pattern generalised from incident, feedback and evaluation loops at Microsoft · public analogues: OpenAI, "Harness engineering" (2026) · Stripe, "Minions" (2026)

LOOP 4 · THE DEVELOPMENT LOOP ITSELF

Intent goes in. A verified pull request comes out.



Anthropic's AI-native SDLC: intent → spec → plan → code → PR

PLATFORM FEATURES, AT MICROSOFT

Spec → human-locked design → PR-sized low-level designs → code. Each feature took **~1.5 to 14 human hours**, roughly **a fifth** of the elapsed time.

A SERVICE REBUILT, AT MICROSOFT

~9 weeks, agents wrote every file. The old system was the **executable spec**. **~1.6** lines of test per line of code.

Before automating a step, **run it by hand** on two real cases

Anthropic, "The AI-native SDLC playbook" (2026) · right: generalised from engineering teams at Microsoft, numbers approximate

WHAT THE FOUR LOOPS HAVE IN COMMON

Generating is one step. Checking is most of the machine.

~14

analysers in the eval machine,
~6 of them just checking the eval

5

critic stages before a finding reaches a human

~2 per PR

findings from a review agent, across hundreds of agent PRs

CRITIC · WHILE GENERATING

Tests and linters in the loop · a reviewer from another model family · a feature checklist to tick

ORACLE · BEFORE IT COUNTS

Hidden tests · the real end state, not the narration · a human for consequential changes

The machine **never approves** its own change.
Every correction **updates the harness**.

The agent can't say it's done. The build has to.

Generalised from engineering teams at Microsoft, numbers approximate · Anthropic, "Effective harnesses for long-running agents" (2025) · Regehr, "Zero-degrees-of-freedom programming"

THE VERDICT

Who decides whether it got smarter or dumber?

Every suspect led back to the same witness.

CONTEXT | THE EVAL | DECISIONS | ARCHITECTURE | TIME | SELF-TUNING

GENERATING GOT CHEAP. VERIFYING DIDN'T.

Speed moved the bottleneck to review and verification

RANDOMISED TRIAL · EXPERIENCED OPEN-SOURCE DEVS

19% slower with AI tools, while believing they were **20% faster**

100K+ DEVELOPERS · COMMIT-LEVEL DATA

Sync agents: **+741%** lines written, but only **+20%** releases

STRIPE · ONE-SHOT CODING AGENTS

1,000+ agent-written PRs merged a week, **every one** reviewed by a human

SAME ENGINEERS, SAME AGENTS

Median PR wait of **~18 hours** in one repo and **~2 minutes** in another, where review rules were automated checks

METR, early-2025 developer RCT (2025) · Demirer, Musolff & Yang, "Writing Code vs. Shipping Code" (2026) · Stripe, "Minions" (2026) · last: generalised from teams at Microsoft

OPEN PROBLEMS

Six areas ripe for research, and industry needs answers

HONEST OPTIMISERS

Skill and harness optimisers whose gains **survive** held-out tasks, noise and cost

VERIFIERS THAT CAN'T BE GAMED

Gates a self-improving agent **can't exploit**, especially without ground truth

MODEL-SPECIFIC CONTEXT

When does a skill or instruction **transfer** across models and harnesses?

LIBRARY HEALTH

Routing, shadowing and pruning as skill libraries grow into the **hundreds**

DECISION PLACEMENT

Which calls move to a **decision model**, and how thresholds stay valid

ATTRIBUTION AT SCALE

Which agent, skill, tool or harness **caused** this failure, automatically

Enterprises especially need progress here. If you're working on any of these, **I'm happy to talk**.

CASE CLOSED

The model rarely got dumber. The harness around it did.

SO MEASURE IT

Every change is a **hypothesis**: compare with and without, above the noise

SO KEEP IT TRUE

Decisions, architecture and context **age**: re-check them like dependencies

SO GOVERN ITS TUNING

Machines can make it smarter, but only as far as the **evaluator** allows

Build the evaluator first. It's the only witness that can tell you which way your agent is ageing.

ONE IDEA TO TAKE HOME

When agents build the software, engineers build the answer key

SPECIFICATION

The context we write

Skills, specs and contracts that say what “right” means, for this model

VERIFICATION

The gates we build

Evals, critics and oracles that decide “done”, before any optimiser does

JUDGMENT

The calls we keep

Which decisions stay human, and who approves the machine's change



TEDxKLH Bachupally · “If AI Does My Work Better, What Do I Actually Do?”

Tezan Sahu, TEDxKLH Bachupally (YouTube)

THANK YOU

Questions welcome

The Curious Case of the Dumber Agent

Tezan Sahu

CONNECT WITH ME

LinkedIn

Low-Pass Filter newsletter

Follow along for more on agents, evals and harnesses

REFERENCES · 1 OF 2

Context, evaluation, decisions and architecture

CONTEXT & SKILLS

- Gloaguen et al., Evaluating AGENTS.md (2026)
- OpenAI, Harness engineering (2026)
- Li et al., SkillsBench (2026)
- Harness Updating Is Not Harness Benefit · Skills at Scale (2026)

EVALUATION

- ASSAY · More Skills, Worse Agents (2026)
- GitHub, Evaluating the Copilot agentic harness (2026)
- Anthropic, Demystifying evals for AI agents (2026)

DECISIONS

- JevAgentHarness · Jev-LiteMAS · JEV-as-a-Judge (2026)
- GitHub, Project HydraFusion (2026)
- Anthropic, Claude Code mods (2026)

ARCHITECTURE & TIME

- Anthropic, Multi-agent research system · Cognition, Don't Build Multi-Agents (2025)
- Cemri et al., MAST · Zhang et al., Who&When (2025)
- GitHub, Fewer tools (2025) · Agentic memory (2026)
- Skill Drift Is Contract Violation (2026) · Anthropic, agent teams

REFERENCES · 2 OF 2

Self-tuning skills and harnesses, verification, my work

SKILL OPTIMISATION

- Agent Skill Evaluation & Evolution survey (2026)
- SkillOpt · SelfEvoSkill · VALVE · BiPA (2026)
- GEPA · ACE (2025) · OPRO · TextGrad

HARNESS OPTIMISATION

- Harness² · Meta-Harness · AHE (2026)
- HarnessX · Adaptive Auto-Harness · Phantom Guardrails (2026)
- ADAS (2024) · Darwin Gödel Machine (2025) · Self-evolving agents survey (2025)

VERIFICATION

- METR, Developer productivity RCT (2025)
- Anthropic, AI-native SDLC playbook (2026) · Stripe, Minions (2026)
- Demirer et al., Writing Code vs. Shipping Code (2026)
- Anthropic, Harnesses for long-running agents (2025)

MY WORK

- Sahu, Your Agent Is Paying Essay Prices to Pick from a List (2026)
- Sahu & Das, *AI Builds the UI — Who Judges It?* (SERI 2026)
- Patent filings (2026): guidance-driven context optimisation · Skill Studio · feedback-driven skill improvement